A D Patel Institute of Technology (ADIT)
CVM University, Vallabh Vidyanagar

DESIGN AND ANALYSIS OF ALGORITHMS PROJECT REPORT

Traffic Shortest Path Finder

Dynamic Route Planning Using Dijkstra's Algorithm

Submitted by

Desai Aryan M.

Enrollment No.: 12402080601009

Department of Information Technology
A D Patel Institute of Technology (ADIT)
CVM University, Vallabh Vidyanagar

INDEX

Topic	Page No.
ABSTRACT	1
1. INTRODUCTION	2
1.1 Background	2
1.2 Motivation	2
1.3 Organisation of the Report	2
2. PROBLEM STATEMENT	3
2.1 Limitations of Existing Approaches	3
3. OBJECTIVES	4
4. SCOPE OF THE PROJECT	4
5. UNDERSTANDING THE SHORTEST PATH PROBLEM	5
5.1 Graph Model of a Road Network	5
5.2 Representation of a Graph	6
5.3 Shortest Path Algorithms Compared	6
6. OVERVIEW OF THE PROJECT	7
7. SYSTEM ARCHITECTURE	7
7.1 Road Network Data	7
7.2 Traffic and Cost Model	8
7.3 Algorithm Engine	8
7.4 Visualisation Layer	8
7.5 Terminal Versions	8
7.6 Network Data	9
8. DIJKSTRA'S ALGORITHM	10
8.1 Idea of the Algorithm	10
8.2 Steps	11
8.3 Pseudocode	11
8.4 Why the Algorithm Is Correct	11
8.5 Complexity Analysis	12
9. WORKING OF THE SYSTEM	13
10. SYSTEM REQUIREMENTS	14
10.1 Functional Requirements	14
10.2 Non-Functional Requirements	14
10.3 Software Requirements	15
11. KEY FEATURES	15
12. TECHNOLOGIES USED	16
13. IMPLEMENTATION DETAILS	16
13.1 Project Files	16
13.2 Structure of the Web Application	17
13.3 The Relaxation Step	17
13.4 Results Returned by the Engine	17
14. TEST SCENARIOS AND RESULTS	17
14.1 Scenario 1: Normal Traffic	18
14.2 Comparison with Other Routes	19
14.3 Scenario 2: Rush-Hour Congestion on the Centre Roads	20
14.4 Tie-Breaking in Scenario 2	20
14.5 Scenario 3: South Bypass	21
14.6 Summary of Results	22
14.7 Distances Between All Pairs	22
14.8 Observations	23
15. APPLICATIONS	23
16. ADVANTAGES	23
17. CHALLENGES AND LIMITATIONS	24
18. FUTURE SCOPE	24
19. CONCLUSION	24
REFERENCES	25

ABSTRACT

Finding the quickest way across a city is a problem that every commuter faces every day. The route with the smallest distance on a map is not always the fastest, because roads become congested, slow down and sometimes block completely. A navigation system that only compares distances cannot react to these changes. This report presents the **Traffic Shortest Path Finder**, a Design and Analysis of Algorithms (DAA) project that applies Dijkstra's shortest path algorithm to a small city road network and re-computes the best route whenever traffic conditions change.

The city is modelled as a weighted graph of eight locations (A to H) joined by fourteen two-way roads. Each road has a length in kilometres and a traffic state, either normal or heavy. A heavy road multiplies the cost of using that road by 2.5, so Dijkstra's algorithm automatically prefers a longer but freer road. The algorithm has been written from scratch, without any library, in three forms: an interactive web application (HTML, CSS and JavaScript), a C program and a Python program for the terminal.

The report explains the problem, the graph model, the design and working of the system, its requirements and features, and the implementation. It then presents test scenarios with full step-by-step execution traces, showing that a jam on the central roads moves the best route from A → D → H (17 km, 20.4 minutes) to A → C → F → H (18 km, 21.6 minutes). In the same scenario, the unchanged route would have taken 51 minutes. The report closes with advantages, limitations, applications and future scope.

Keywords: Dijkstra's Algorithm, Shortest Path, Weighted Graph, Traffic Congestion, Dynamic Rerouting, Greedy Algorithm, Route Visualisation

1. INTRODUCTION

1.1 Background

Road networks in cities can be viewed as graphs. Junctions and important places become vertices, and the roads between them become edges. Once a road network is described in this way, the question “what is the best way to get from here to there?” becomes the well-known shortest path problem of graph theory.

In 1959 Edsger W. Dijkstra published a simple and efficient algorithm that solves this problem for graphs whose edge weights are not negative. The algorithm is still taught in every algorithms course, and variants of it are used inside modern map and routing software. It is a classic example of a greedy algorithm: at each step it settles the closest unsettled vertex and never has to reconsider that choice.

The Traffic Shortest Path Finder uses this algorithm to solve a practical version of the problem. Besides the length of each road, it also takes the current traffic condition into account, so that the answer changes when the traffic changes.

1.2 Motivation

A route that is shortest on paper may be the slowest in practice during rush hour. Drivers need a tool that looks at the cost of travelling, which combines distance and congestion, and not only at the distance. For students, the algorithm itself is often hard to follow when it is only seen as pseudocode. The motivation for this project is therefore twofold: to show how a shortest path algorithm reacts to changing road conditions, and to make each step of the algorithm visible, so that its greedy working can be understood by watching it run.

1.3 Organisation of the Report

Chapter 2 states the problem and Chapter 3 lists the objectives. Chapter 4 defines the scope. Chapter 5 explains the graph model and compares shortest path algorithms, and Chapters 6 and 7 describe the project and its architecture. Chapter 8 explains Dijkstra’s algorithm in detail and Chapter 9 describes the working of the system. Chapters 10 to 12 cover requirements, features and technologies, and Chapter 13 describes the implementation. Chapter 14 presents the test scenarios and results. Chapters 15 to 17 discuss applications, advantages and limitations, and Chapters 18 and 19 give the future scope and conclusion.

2. PROBLEM STATEMENT

Route planning on a city road network faces three main problems:

1. **Distance is not time:** The geographically shortest route is not always the fastest, because congested roads slow down traffic.
2. **Static routes:** A route that was calculated once does not adapt when a road becomes jammed, so the driver keeps following a route that is no longer good.
3. **Hidden algorithm:** In most navigation tools the algorithm works as a black box. A learner cannot see which places were examined, in what order, and why one route was chosen over another.

The problem can be stated formally as follows. Given a road network represented as a graph G = (V, E), where V is a set of eight locations (A to H) and E is a set of fourteen two-way roads, each road (u, v) has a physical length d(u, v) in kilometres and a traffic multiplier T(u, v), equal to 1.0 for normal traffic and 2.5 for heavy traffic. The effective weight of a road is w(u, v) = d(u, v) × T(u, v). For a chosen source S and destination D, find the path from S to D for which the sum of effective weights is the smallest, and find it again whenever the traffic state of any road changes.

2.1 Limitations of Existing Approaches

A printed or static map shows only distances and cannot show current conditions. A simple distance-based search finds the shortest route in kilometres, but it ignores congestion. Full navigation applications handle live traffic, but they are large systems with proprietary data and they do not show the working of the algorithm to the user. There is a need for a small, open and transparent system that finds the best route under changing traffic and also explains how it found it.

3. OBJECTIVES

The main objectives of the project are:

- To model a city road network as a weighted, undirected graph of eight locations and fourteen roads.
- To implement Dijkstra's shortest path algorithm from scratch, without using any graph or routing library.
- To represent traffic congestion through an effective cost multiplier and to re-compute the best route when traffic changes.
- To visualise the road network, the selected route and the traffic state in an interactive web interface.
- To display the execution trace of the algorithm, showing the order in which nodes are settled and how distances are updated.
- To provide the same algorithm as a C program and a Python program that run in a terminal.
- To analyse the time and space complexity of the algorithm and compare it with other shortest path algorithms.

4. SCOPE OF THE PROJECT

The project is a teaching and demonstration tool for the shortest path problem under changing traffic. Table 4.1 shows what is included in the scope of this report and what is left for future work.

Table 4.1: Scope of the project

Aspect	Description
Included	Graph model, Dijkstra's algorithm, traffic cost model, web visualisation, terminal programs, test scenarios and complexity analysis.
Network	A fixed network of 8 locations (A to H) and 14 two-way roads with distances of 3 to 9 km.
Traffic model	Two states per road: normal (50 km/h, multiplier 1.0) and heavy (20 km/h, multiplier 2.5), set by the user.
Users	Students, teachers and anyone who wants to see how a shortest path algorithm works.
Not included	Live traffic feeds, GPS data, real city maps, one-way roads and turn restrictions, and networks with thousands of nodes.

5. UNDERSTANDING THE SHORTEST PATH PROBLEM

5.1 Graph Model of a Road Network

A graph consists of vertices and edges. In the road network, every junction or landmark is a vertex and every road between two of them is an edge. Each edge carries a weight, which is the cost of travelling along it. Figure 5.1 shows how the real road network is converted into a weighted graph and how the result of the algorithm returns to the commuter as a route. Table 5.1 shows how each part of the real network is represented in the project.

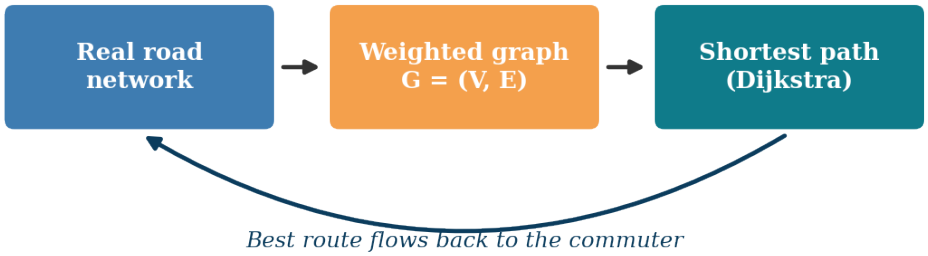


Figure 5.1: From road network to shortest path

Table 5.1: Mapping of the road network to the graph

Real world	Graph element	Representation in the project
Junction or landmark	Vertex	Nodes A to H, such as Central Terminus (A) and Airport Terminal (H)
Road between two places	Edge (undirected)	14 roads, each usable in both directions
Length of the road	Base weight d	Distance in kilometres (3 to 9 km)
Traffic condition	Multiplier T	1.0 for normal traffic, 2.5 for heavy traffic
Cost of using the road	Effective weight w	w = d × T

5.2 Representation of a Graph

A graph can be stored in the computer in two common ways, as shown in Table 5.2. The web version of the project builds an adjacency list every time a route is requested, so that each node knows its neighbours and the effective weight of the road to each of them. The C and Python versions build an adjacency (cost) matrix of size 8 × 8, which is simple and fast for a graph of this size.

Table 5.2: Adjacency matrix and adjacency list

Property	Adjacency matrix	Adjacency list
Space needed	$O(V^2)$	$O(V + E)$
Check if a road exists	$O(1)$	O(degree of the node)
List all neighbours of a node	$O(V)$	O(degree of the node)
Best suited for	Dense graphs and small graphs	Sparse graphs, such as real road networks
Used in this project	C and Python programs	Web application

5.3 Shortest Path Algorithms Compared

Several algorithms solve shortest path problems. They differ in the kind of graph they accept and in their running time. Table 5.3 compares the most common ones. Dijkstra's algorithm was chosen because the road weights are never negative, because only one source is needed at a time, and because it is the algorithm that is used as the basis of real routing systems.

Table 5.3: Comparison of shortest path algorithms

Algorithm	Graph type	Time complexity	Remark
Breadth-First Search	Unweighted	$O(V + E)$	Counts roads, not distance; cannot use weights
Dijkstra	Non-negative weights	$O(V^2)$ or $O((V + E) \log V)$	Single source; greedy; fast
Bellman-Ford	Negative weights allowed	$O(V \times E)$	Slower; detects negative cycles
Floyd-Warshall	Any, all pairs	$O(V^3)$	Gives all pairs at once; heavy for large graphs
A*	Non-negative, with coordinates	Depends on heuristic	Dijkstra guided by an estimate of remaining distance

6. OVERVIEW OF THE PROJECT

The Traffic Shortest Path Finder is a route planning tool for a small city network. The user chooses a starting location and a destination, marks some roads as congested, and the system shows the best route, its length, its estimated travel time and the steps that the algorithm followed. Its main ideas are:

- It models the city as a weighted graph of 8 locations and 14 roads.
- It uses an effective cost, which is the distance multiplied by a traffic factor, instead of the distance alone.
- It runs Dijkstra's algorithm, written from scratch, and re-runs it each time the traffic or the selection changes.
- It shows the network, the highlighted route and the algorithm trace in an interactive page, and in a terminal for the C and Python versions.

The same network, the same cost model and the same algorithm are present in all three implementations (`script.js`, `dijkstra.c` and `terminal_runner.py`). This makes it possible to check that all of them give the same answers.

7. SYSTEM ARCHITECTURE

The system works as a pipeline of five stages, from defining the road network to showing the result, as shown in Figure 7.1.



Figure 7.1: Processing pipeline of the Traffic Shortest Path Finder

7.1 Road Network Data

The network is stored as two simple data structures. The first is the list of nodes, each with a letter, a place name and a position for drawing. The second is the list of roads, each with two end nodes, a distance in kilometres and a traffic flag (`isHeavy`). All other parts of the system read from these two lists.

7.2 Traffic and Cost Model

Every road has one of two states. A normal road allows an average speed of 50 km/h and has a multiplier of 1.0. A heavy road allows only 20 km/h and has a multiplier of 2.5. The effective weight used by the algorithm is

$$w(u, v) = d(u, v) \times T(u, v)$$

The multiplier 2.5 is the same as the ratio of the two speeds (50 ÷ 20). This means that the effective weight is always proportional to the time needed to drive the road, so the route with the smallest total effective weight is also the route with the smallest travel time. The travel time shown to the user is calculated separately as the sum of distance ÷ speed for each road on the route.

7.3 Algorithm Engine

The engine takes a source and a destination, builds the cost structure from the current traffic state, runs Dijkstra's algorithm and returns the route, the total distance, the effective cost, the travel time, the order in which nodes were settled and a complete trace of every step. The engine does not depend on the user interface, which is why the same logic could be written in JavaScript, C and Python.

7.4 Visualisation Layer

The web page draws the network as an SVG image made of four layers placed one over the other: the road lines, the distance labels, the highlighted shortest route and the nodes. Clicking a road toggles its traffic state, and clicking a node selects it as the destination. After each change, the route is recalculated and redrawn. The results panel shows the route as a chain of nodes, together with the total distance, the effective cost, the travel time and the settled order.

7.5 Terminal Versions

The C program and the Python program provide a text menu with options to find a route between any two locations, toggle the traffic on a road, run two preset demonstrations and view the road table. Both print an execution trace table for every search. A small launcher script (`Run_In_Terminal.command`) compiles and runs the C program if a compiler is available and otherwise falls back to the Python program.

7.6 Network Data

Table 7.1 lists the eight locations and Table 7.2 lists the fourteen roads. The sum of the node degrees in Table 7.1 is 28, which is twice the number of roads, as it must be for an undirected graph. City Center (D) is the busiest junction, with seven roads.

Table 7.1: Locations in the network

Node	Location	Degree	Directly connected to
A	Central Terminus	3	B, C, D
B	North Tech Hub	3	A, D, E
C	South Market	3	A, D, F
D	City Center	7	A, B, C, E, F, G, H
E	Industrial Zone	3	B, D, G
F	Metro Plaza	3	C, D, H
G	University & Medical Hub	3	D, E, H
H	Airport Terminal	3	D, F, G

Table 7.2: Roads in the network

ID	Road	Length (km)	ID	Road	Length (km)
1	A – B	5	8	D – E	3
2	A – C	6	9	D – F	3
3	A – D	8	10	E – G	4
4	B – E	4	11	D – G	6
5	B – D	4	12	F – H	8
6	C – D	4	13	G – H	5
7	C – F	4	14	D – H	9

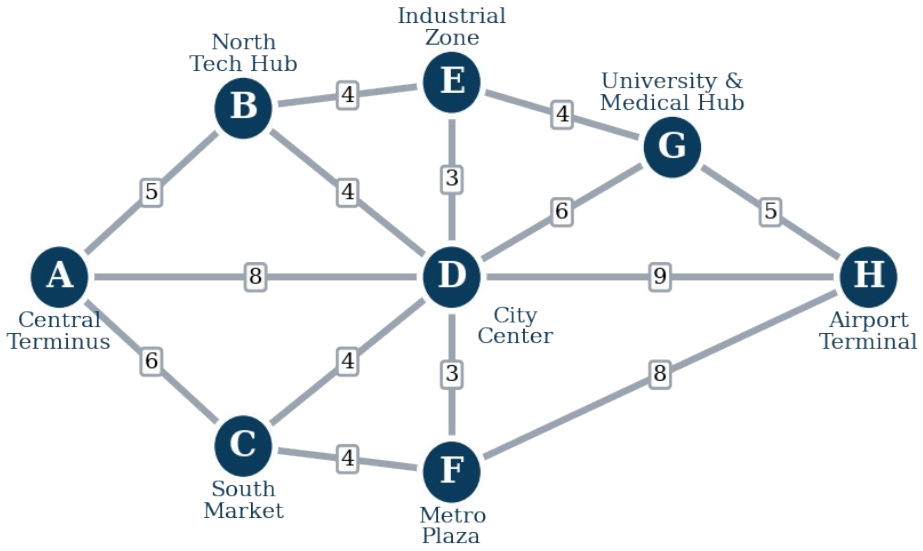


Figure 7.2: The 8-node, 14-road network (numbers are distances in km)

8. DIJKSTRA'S ALGORITHM

8.1 Idea of the Algorithm

Dijkstra's algorithm keeps, for every node, the best known cost of reaching it from the source. At the start only the source is known, with cost 0. The algorithm repeatedly takes the unsettled node with the smallest known cost, declares that cost final, and checks whether going through this node gives a cheaper way to reach any of its neighbours. When the destination is settled, its cost is the answer. The route is recovered by following the predecessor (parent) of each node back to the source.

8.2 Steps

1. **Initialisation:** Set dist[source] = 0 and dist[v] = ∞ for every other node. Set parent[v] to none and visited[v] to false for all nodes.
2. **Greedy selection:** Among the nodes that are not yet visited, choose the node u with the smallest dist[u]. If none has a finite distance, stop; the remaining nodes are unreachable.
3. **Settle:** Mark u as visited. If u is the destination, stop.
4. **Edge relaxation:** For every unvisited neighbour v of u, compute tentative = dist[u] + w(u, v). If tentative < dist[v], set dist[v] = tentative and parent[v] = u.
5. **Repeat** from step 2.
6. **Path reconstruction:** Starting at the destination, follow parent pointers back to the source and reverse the list.

8.3 Pseudocode

```
DIJKSTRA(G, w, source, dest)
  for each node v in V:
      dist[v] = INF;  parent[v] = NIL;  visited[v] = false
  dist[source] = 0
  repeat:
      u = unvisited node with minimum dist[u]
      if no such node or dist[u] == INF:  break
      visited[u] = true
      if u == dest:  break
      for each neighbour v of u with visited[v] == false:
          if dist[u] + w(u, v) < dist[v]:
              dist[v]   = dist[u] + w(u, v)
              parent[v] = u
  return dist[dest], path built from parent[]
```

8.4 Why the Algorithm Is Correct

When a node u is selected, its distance is the smallest among all unsettled nodes. Any other route to u would have to pass through some unsettled node first, and that node is already at least as far away as u. Because no road has a negative cost, the rest of such a route cannot make the total smaller. The distance of u is therefore final. This argument fails if negative weights are allowed, which is why Bellman-Ford must be used in that case. In this project all weights are positive (distance × 1.0 or × 2.5), so the condition is always satisfied.

8.5 Complexity Analysis

Table 8.1 gives the complexity of the algorithm. The project selects the minimum node by scanning all nodes, which takes O(V) for each of the V selections, so the running time is $O(V^2)$ for the matrix version and $O(V^2 + E)$ for the web version, which uses an adjacency list. With V = 8 and E = 14, a search finishes in well under a millisecond. For large road networks, an adjacency list with a binary min-heap (priority queue) reduces the time to O((V + E) log V).

Table 8.1: Time and space complexity

Implementation	Time	Space	Used in
Array scan + cost matrix	$O(V^2)$	$O(V^2)$	C, Python
Array scan + adjacency list	$O(V^2 + E)$	O(V + E)	Web (JavaScript)
Adjacency list + binary min-heap	O((V + E) log V)	O(V + E)	Large networks (not needed here)

9. WORKING OF THE SYSTEM

The working of the system can be understood in the following steps:

1. The user selects a starting location and a destination from the two drop-down lists. The system rejects the selection if both are the same.
2. The user optionally marks some roads as heavy traffic by clicking them or by using the traffic list. A preset can also be loaded for a quick demonstration.
3. The engine calculates the effective weight of every road from its distance and its traffic state.
4. Dijkstra's algorithm runs and records every step in a trace.
5. The route is reconstructed from the parent pointers, and the total distance, effective cost and travel time are calculated.
6. The page highlights the route on the network and fills the results panel. If the destination cannot be reached, a message is shown instead.
7. Whenever the user changes the traffic or the selection, steps 3 to 6 are repeated automatically.

Figure 9.1 shows this process as a flowchart. The cycle repeats each time the user changes the traffic or the route.

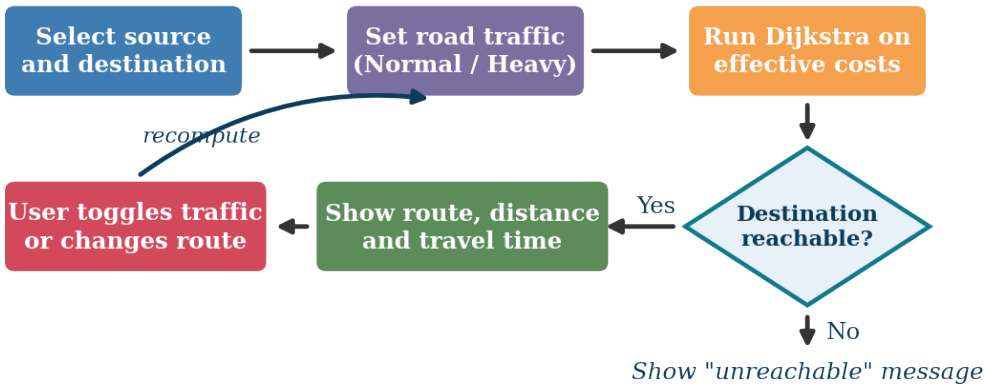


Figure 9.1: Working flow of the system

10. SYSTEM REQUIREMENTS

10.1 Functional Requirements

Functional requirements describe what the system must do. They are listed in Table 10.1.

Table 10.1: Functional requirements

No.	Requirement
F1	The system shall let the user choose any two different locations from A to H as source and destination.
F2	The system shall find the route of minimum effective cost using Dijkstra's algorithm.
F3	The system shall let the user mark any road as normal or heavy traffic.
F4	The system shall re-calculate the route automatically when the traffic or the selection changes.
F5	The system shall display the route, total distance, effective cost and estimated travel time.
F6	The system shall display the order in which nodes were settled and the steps of the algorithm.
F7	The system shall provide preset demonstrations and a reset option.
F8	The system shall report an error when the source and destination are the same, or when the destination is unreachable.

10.2 Non-Functional Requirements

Non-functional requirements describe how well the system must work. They are listed in Table 10.2.

Table 10.2: Non-functional requirements

Quality	Requirement
Speed	A route must be calculated and drawn instantly, so that the display feels real-time.
Correctness	All three implementations must give the same route and cost for the same traffic state.
Usability	The interface must be simple enough for a student to use without instructions.
Portability	The web version must run in any modern browser without installation or a server.
Maintainability	The network data must be separate from the algorithm so that nodes or roads can be changed easily.

10.3 Software Requirements

- **Web version:** any modern browser such as Chrome, Safari or Edge. No server, database or installation is needed.
- **C version:** a C compiler that supports the C99 standard, for example `gcc dijkstra.c -o dijkstra`.
- **Python version:** Python 3, with no extra packages.

11. KEY FEATURES

The main features of the project are described below.

- **Interactive road network:** The eight locations and fourteen roads are drawn as an SVG graph with distance labels on every road.
- **Traffic simulator:** Any road can be switched between normal and heavy traffic. Roads in heavy traffic are shown in red and dashed.
- **Automatic rerouting:** The shortest route is re-calculated as soon as the traffic changes, and is drawn as a thick green line over the network.
- **Complete route breakdown:** The route is shown as a chain of nodes and road legs, with total distance, effective cost and estimated travel time.
- **Algorithm trace:** The order in which nodes are settled and the relaxation of each neighbour are recorded, so the greedy working can be followed.
- **Preset demonstrations:** Three ready-made scenarios (normal traffic, rush-hour rerouting and south bypass) show the behaviour of the system in one click.
- **Input validation:** The system does not accept the same node as source and destination.
- **Terminal versions:** The same engine in C and Python with a text menu and printed trace tables.

The traffic simulator is the key feature, because it turns a textbook algorithm into something that can be tested by the user. The trace shows why a node was chosen, and the presets make it easy to demonstrate the main idea of the project in a few seconds.

12. TECHNOLOGIES USED

The project deliberately uses only basic technologies, so that it can run anywhere and the algorithm is not hidden behind a library. The technologies are listed in Table 12.1.

Table 12.1: Technologies for each layer

Layer	Technology
Page structure	HTML5 with semantic elements
Styling	CSS3 with Grid, Flexbox and custom properties (dark-blue glass style)
Algorithm and logic	JavaScript (ES6), written from scratch with no libraries
Visualisation	Scalable Vector Graphics (SVG) with a glow filter for the route
Terminal version	C (C99 standard) and Python 3
Launcher	Shell script that compiles the C program or falls back to Python
Fonts	Inter and JetBrains Mono from Google Fonts (the page still works offline with default fonts)

13. IMPLEMENTATION DETAILS

13.1 Project Files

The project consists of seven files, listed in Table 13.1 with their sizes.

Table 13.1: Files of the project

File	Lines	Purpose
index.html	227	Page layout: controls, network canvas, traffic list and results panel
style.css	1249	Appearance, layout, colours and responsive design
script.js	639	Network data, Dijkstra’s algorithm, drawing, traffic control and presets
dijkstra.c	346	Terminal version in C with menu, trace table and presets
terminal_runner.py	217	Equivalent terminal version in Python
Run_In_Terminal.command	13	Launcher that tries C first and then Python
README.md	138	Description, topology, sample demonstrations and viva questions

13.2 Structure of the Web Application

In `script.js` the code is arranged in seven parts: the network data (`NODES`, `ROADS` and the speed constants), the `dijkstra()` function, the SVG drawing functions (`renderGraph`, `highlightPath`), the traffic control functions (`toggleRoadTraffic`, `setRoadTraffic`), the search and result functions (`executeRouteSearch`, `updateResultsUI`), the preset loader (`loadPreset`) and the event listeners that start everything when the page loads.

13.3 The Relaxation Step

The core of the algorithm is the relaxation of the neighbours of the selected node. The code below is the relaxation loop of the web version, shortened to its main lines.

```
for (const edge of adjList[minNode]) {
  const neighbor = edge.neighbor;
  if (!visited[neighbor]) {
    const tentativeDist = dist[minNode] + edge.effectiveWeight;
    if (tentativeDist < dist[neighbor]) {
      dist[neighbor]   = tentativeDist;
      parent[neighbor] = minNode;
    }
  }
}
```

The effective weight used above is created while the adjacency list is built, with `distance × (isHeavy ? 2.5 : 1.0)`. The C and Python versions use the same expression through the function `getEffectiveCost()` / `get_cost()`.

13.4 Results Returned by the Engine

The function returns the path, the legs of the path, the total distance, the effective cost, the travel time in minutes, the settled order, the full trace log and a flag that says whether the destination was reachable. The user interface only displays these values, which keeps the algorithm separate from the drawing code.

14. TEST SCENARIOS AND RESULTS

Three scenarios were run on the network. All values below were obtained by running the algorithm of the project and were cross-checked with the Python terminal version (`terminal_runner.py`). In every scenario the source is A (Central Terminus) and the destination is H (Airport Terminal). The roads of the network are shown in Figure 7.2.

14.1 Scenario 1: Normal Traffic

All roads are clear. Table 14.1 shows the execution trace. Node A is settled first with cost 0 and relaxes B, C and D. Node D is settled at cost 8, and it offers the direct road D – H, which gives H a cost of 17. No other node can reach H more cheaply, so H is settled at cost 17.

Table 14.1: Execution trace, Scenario 1 (normal traffic)

Step	Settled node	Cost	Costs updated (node:cost)
1	A (Central Terminus)	0	B:5, C:6, D:8
2	B (North Tech Hub)	5	E:9
3	C (South Market)	6	F:10
4	D (City Center)	8	G:14, H:17
5	E (Industrial Zone)	9	G:13
6	F (Metro Plaza)	10	No updates
7	G (University & Medical Hub)	13	No updates
8	H (Airport Terminal)	17	Destination settled; search stops

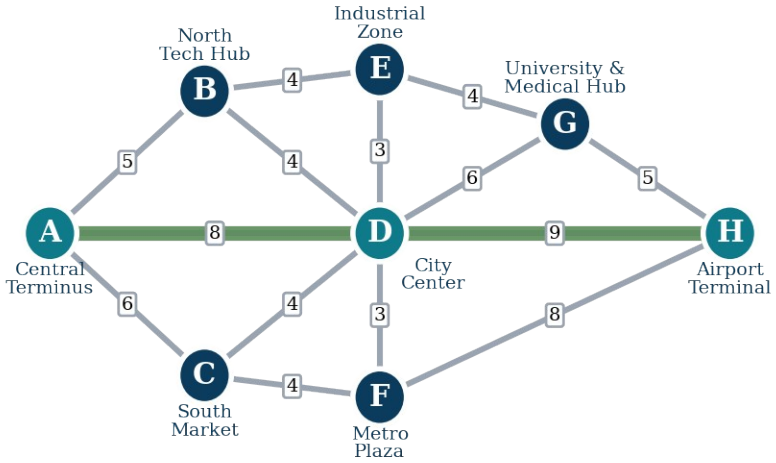


Figure 14.1: Scenario 1, shortest route A → D → H

The result is the route **A → D → H** with a distance of **17 km**, an effective cost of 17 and an estimated travel time of **20.4 minutes** at 50 km/h. The other main routes are compared in section 14.2.

14.2 Comparison with Other Routes

Table 14.2 lists the cost of the main alternative routes under normal traffic. No route is cheaper than A → D → H, and three different routes tie for second place at 18 km.

Table 14.2: Main candidate routes, Scenario 1

Route	Cost (km)	Remark
A → D → H	17	Shortest (selected)
A → B → D → H	18	1 km longer
A → B → E → G → H	18	North corridor
A → C → F → H	18	South corridor
A → C → D → H	19	
A → D → G → H	19	

14.3 Scenario 2: Rush-Hour Congestion on the Centre Roads

The roads A – D and D – H are marked as heavy traffic. Their effective costs rise from 8 to 20 and from 9 to 22.5. The old route A → D → H would now cost 42.5. Table 14.3 shows the new trace.

Table 14.3: Execution trace, Scenario 2 (A – D and D – H heavy)

Step	Settled node	Cost	Costs updated (node:cost)
1	A (Central Terminus)	0	B:5, C:6, D:20
2	B (North Tech Hub)	5	D:9, E:9
3	C (South Market)	6	F:10
4	D (City Center)	9	G:15, H:31.5
5	E (Industrial Zone)	9	G:13
6	F (Metro Plaza)	10	H:18
7	G (University & Medical Hub)	13	No updates
8	H (Airport Terminal)	18	Destination settled; search stops

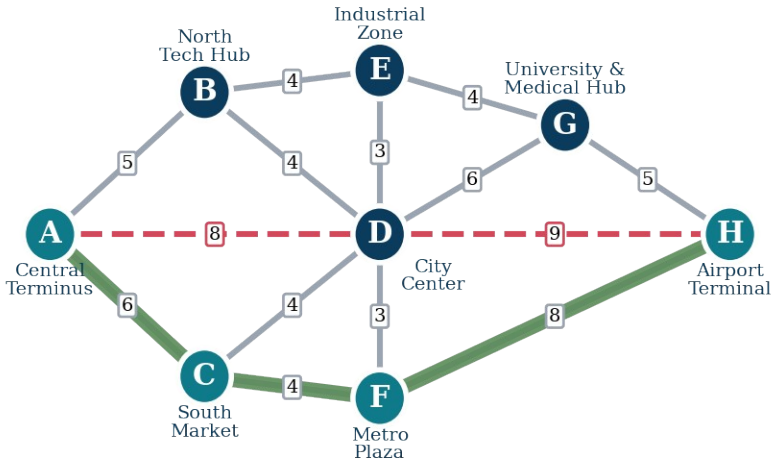


Figure 14.2: Scenario 2, rerouted shortest path A → C → F → H

The route changes to **A → C → F → H** with a distance of **18 km** and an estimated travel time of **21.6 minutes**. The system chose a road that is 1 km longer but avoids the jam completely.

14.4 Tie-Breaking in Scenario 2

In this scenario the north corridor A → B → E → G → H also has a cost of exactly 18 (5 + 4 + 4 + 5). Dijkstra's algorithm finds one of the two equal routes, depending on the order in which nodes are examined. In this implementation F is settled before G, so F reaches H first with cost 18, and G cannot improve it because 18 is not smaller than 18. The route A → C → F → H is therefore reported. Both routes are equally good, but the comment in the source code and the message printed by the C and Python demos mention the north route, while the program actually returns the south route.

14.5 Scenario 3: South Bypass

The roads A – B and A – D are marked as heavy traffic, so two of the three roads leaving A are congested. Table 14.4 shows the trace. Node C is settled second, and its neighbour F gives the cheapest way to H.

Table 14.4: Execution trace, Scenario 3 (A – B and A – D heavy)

Step	Settled node	Cost	Costs updated (node:cost)
1	A (Central Terminus)	0	B:12.5, C:6, D:20
2	C (South Market)	6	D:10, F:10
3	D (City Center)	10	E:13, G:16, H:19
4	F (Metro Plaza)	10	H:18
5	B (North Tech Hub)	12.5	No updates
6	E (Industrial Zone)	13	No updates
7	G (University & Medical Hub)	16	No updates
8	H (Airport Terminal)	18	Destination settled; search stops

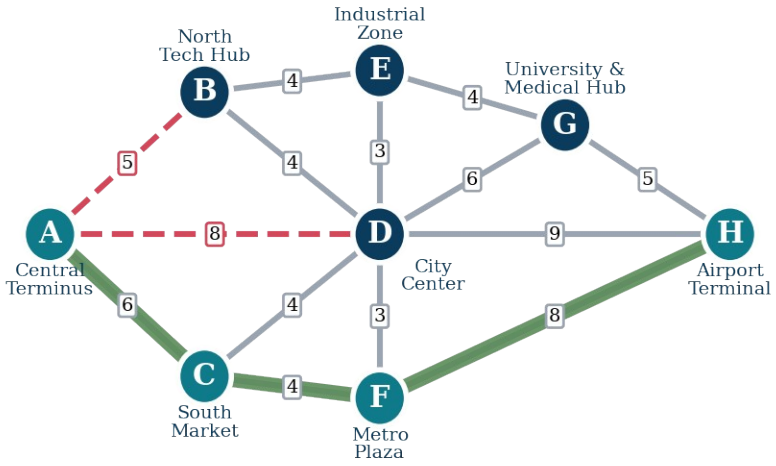


Figure 14.3: Scenario 3, south bypass A → C → F → H

The route is again **A → C → F → H**, 18 km long, with a travel time of **21.6 minutes**. The alternative through the north corridor would cost 25.5 and the road through D would cost 29.

14.6 Summary of Results

Table 14.5 summarises the three scenarios. The last two columns compare the selected route with the route that would have been chosen if the traffic had been ignored. In Scenario 2, ignoring the traffic would have kept the driver on A → D → H, where both roads are jammed, and the trip would take 51 minutes instead of 21.6 minutes. In Scenario 3, only A – D is jammed on that route, and the saving is 13.2 minutes.

Table 14.5: Summary of the three scenarios (A to H)

Scenario	Selected route	Distance (km)	Cost	Time (min)	Time if the old route A → D → H were kept
1. Normal	A → D → H	17	17.0	20.4	20.4 min (no change needed)
2. Centre jammed	A → C → F → H	18	18.0	21.6	51.0 min (saving 29.4 min)
3. South bypass	A → C → F → H	18	18.0	21.6	34.8 min (saving 13.2 min)

14.7 Distances Between All Pairs

Running the engine for every pair of locations under normal traffic gives Table 14.6. The matrix is symmetric, as expected for two-way roads. The largest value is 17 km, between A and H, so this pair is the farthest apart in the network, and it is the pair used in all the demonstrations.

Table 14.6: Shortest distances (km) between all pairs, normal traffic

	A	B	C	D	E	F	G	H
A	0	5	6	8	9	10	13	17
B	5	0	8	4	4	7	8	13
C	6	8	0	4	7	4	10	12
D	8	4	4	0	3	3	6	9
E	9	4	7	3	0	6	4	9
F	10	7	4	3	6	0	9	8
G	13	8	10	6	4	9	0	5
H	17	13	12	9	9	8	5	0

14.8 Observations

- The route changes only when the congestion makes the old route more expensive than an alternative; a jam on a road that is not on the route has no effect on the answer.
- A longer route in kilometres can be the better route in time, which is the main idea that the project demonstrates.
- Equal-cost routes can exist, and the one that is reported depends on the order in which nodes are examined.
- The README describes an earlier 6-node version of the network. The code, the web page and the terminal programs all use the 8-node, 14-road network described in this report.

15. APPLICATIONS

The same method, on much larger graphs, is used in many real systems:

- **Navigation and maps:** Finding the fastest driving, walking or cycling route with live traffic.
- **Emergency services:** Routing ambulances, fire engines and police vehicles around closed or crowded roads.
- **Delivery and logistics:** Planning the cheapest routes for couriers, buses and goods vehicles.
- **Computer networks:** Link-state routing protocols such as OSPF use Dijkstra's algorithm to find the best path for data packets.
- **Games and robotics:** Finding paths through maps and environments.
- **Public transport planning:** Finding the quickest connection between stops with changing travel times.

16. ADVANTAGES

- Finds the best route for the current traffic and not only the shortest distance.
- Reacts instantly to changes in traffic through automatic recalculation.
- Shows every step of the algorithm, which makes it useful for learning and teaching.
- Needs no installation, server or external library for the web version.
- Gives the same results in JavaScript, C and Python, so the logic can be checked in three ways.
- Runs in well under a millisecond for the network used.

17. CHALLENGES AND LIMITATIONS

Table 17.1 lists the main limitations of the project together with ways to reduce them.

Table 17.1: Limitations and possible solutions

Limitation	Description	Possible solution
Fixed small network	Only 8 nodes and 14 roads; no real map data.	Load real road data, for example from OpenStreetMap.
Manual traffic	Traffic is set by the user and is not read from sensors or live feeds.	Connect to a traffic data service.
Two traffic levels	Only normal and heavy; real congestion varies continuously.	Use a continuous multiplier or time-of-day profiles.
Two-way roads only	One-way streets, turn restrictions and signals are not modelled.	Use a directed graph with turn costs.
Linear minimum search	The $O(V^2)$ selection is slow for very large networks.	Use a binary min-heap or a Fibonacci heap.
Tie handling	When two routes cost the same, only one is shown.	List all equal-cost routes or let the user choose a rule.

18. FUTURE SCOPE

- Using a priority queue to make the algorithm scale to city-sized graphs.
- Reading real road maps and live traffic data to replace the fixed network.
- Adding A* search and comparing its explored nodes with Dijkstra’s algorithm on the same map.
- Supporting one-way roads, road closures and time-dependent travel times.
- Showing the algorithm as an animation, step by step, in the web page.
- Adding several stops in one trip, and alternative routes ranked by time.

19. CONCLUSION

The Traffic Shortest Path Finder shows how Dijkstra’s algorithm can be used to find the best route in a road network whose conditions change. By multiplying the length of a congested road by 2.5, the project makes the algorithm look at travel time rather than distance alone, and the tests show that the route changes exactly when the congestion makes another route cheaper. In the main test, a jam on the centre roads moves the route from 17 km to 18 km, and cuts the travel time from 51 minutes to 21.6 minutes.

The project also makes the algorithm easy to understand, because every step can be seen in the trace, and because the same logic is available in a browser, in C and in Python. Although the network is small and the traffic is entered by hand, the method is the same as in real navigation systems, and the project can be extended with larger maps, live data and faster data structures.

REFERENCES

1. E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
3. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, Pearson.
4. J. Kleinberg and É. Tardos, *Algorithm Design*, Pearson.
5. R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.
6. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice Hall.
7. MDN Web Docs, “SVG: Scalable Vector Graphics,” Mozilla Developer Network.